

Questions for participants

Section 1: Demographics & Experience

To be completed before starting the programming task.

1. **Current Status:** Are you currently a student or working in the industry? Yes
 2. **Field of Study / Job Title:** What is your major or current role? Graduated
 3. **Professional Experience:** How many years of professional experience do you have (including internships or full-time work)? 4
 4. **LLM Familiarity:** On a scale of 1–5, how frequently do you use AI tools (e.g., ChatGPT, Claude) for coding or software tasks? 2
 5. **Requirements Engineering (RE) Experience:** Have you taken a course in RE or used formal specifications? If so, are you familiar with **IEEE 29148** or **User Story** structures? Yes
-

Section 2: Post-Task Evaluation

To be completed after receiving the AI-generated code.

1. **Feature Coverage:** Describe the extent to which the AI-generated code implemented the functionality. Were there specific features missing?

Yes, the AI-generated code implemented most of the core functionality required for the safety app. It includes request creation, helper registration, assignment logic, handling multiple concurrent users, and basic privacy protection. However, some features were missing, such as real-time communication (e.g., chat), location-based matching (distance calculation), prioritization of urgent requests, and a more advanced notification system.

2. **Coverage Gaps:** If the code did not cover all requirements, why do you think that happened? (e.g., prompt too vague, AI limitations, misinterpretation)

The gaps likely occurred because the prompt, while detailed, still described a high-level concept rather than strict technical requirements. As a result, the AI focused on building a solid backend logic but did not include more advanced or system-level features like real-time messaging, GPS-based matching, or user authentication. These typically require integration with external services or more explicit instructions.

3. **Coverage Confidence:** What specific evidence in the generated code gives you confidence (or lack thereof) that the requirements were met?

There is good confidence that the core requirements were met because:

The system handles multiple users safely using thread locking (`threading.Lock`), which prevents conflicts.

Helpers are protected from overload through a `max_capacity` constraint.

Requests cannot be double-assigned, ensuring consistency.

Sensitive student data is not exposed to helpers.

However, confidence is lower for real-world usability because features like communication, live updates, and precise matching are not implemented.

4. **Estimated Coverage:** What percentage of the requirements do you believe were successfully covered by the AI-generated code? (___ %)

75

5. **Requirement / Vision Impact:** Which specific part of the Vision (or requirement for Group B) was most helpful in guiding the AI to handle the assigned task correctly?

The most helpful part of the vision was the emphasis on:

- Safety and privacy (led to limiting shared data)
- Handling multiple users simultaneously (led to thread-safe design)
- Avoiding helper overload (led to capacity constraints)

These directly shaped the architecture of the solution.

6. **Challenges:**

- **Intuitive:** Did you find the vision text too broad or unclear for writing an effective AI prompt? Please describe.

The vision was somewhat broad and conceptual, which made it harder to translate directly into specific technical features. For example, “safe communication” could mean many things (encryption, chat, anonymity), and without clarification, only a basic interpretation was implemented.

- **REQC-Guided:** Did you encounter difficulties translating the vision into structured requirements? Please describe.

Yes, there were some difficulties translating the vision into structured requirements. It required making assumptions about:

- What “safe communication” technically includes
- How matching should be prioritized
- What level of detail to include for privacy

Because of this, some features were simplified or omitted in favor of a clean, functional core system.